Continuous integration

GitHub Actions runs the same restore, lint, and coverage commands that can be run on a workstation. The workflow file is `.github/workflows/ci.yml`. The job is named `build` and the workflow is named `ci`.

When GitHub runs it

A run starts for every push, including a push to the default branch, and for every pull request. GitHub checks out that commit on a fresh `ubuntu-latest` machine. Nothing from a previous run is kept on the machine. The coverage report is the only file kept afterwards, and it is stored as a downloadable artifact, not as a commit.

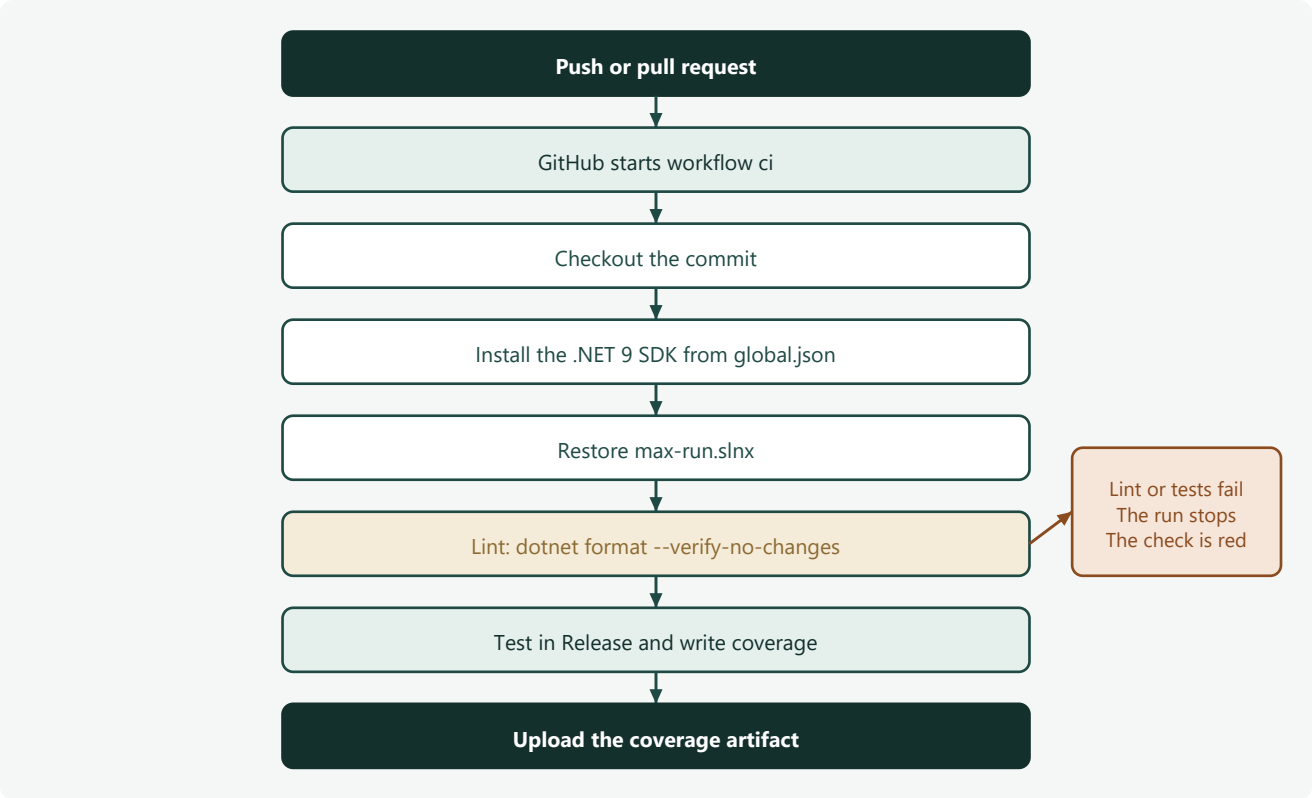


Figure 1. GitHub starts the build job after a push or pull request. A failed lint or test stops the run.

What each step does

Step	What happens	If it fails
Checkout	actions/checkout@v4 copies the commit onto the runner.	The run stops. Later steps do not start.
Install .NET SDK	actions/setup-dotnet@v4 reads global.json and installs SDK 9.0.319, or a newer 9.x SDK.	The run stops before restore.
Restore	dotnet restore max-run.slnx downloads the test packages.	The run stops. Lint and tests are skipped.
Lint	dotnet format checks the tree against EditorConfig and changes nothing.	The check on the commit turns red. The diff that format would have made is in the step log.
Test with coverage	Release tests run. coverlet writes a Cobertura report for the LongestIncreasingRun assembly.	A failed test or a build warning stops the run. The log names the test.
Upload coverage	The coverage directory is saved as an artifact named coverage.	The step fails if that directory is missing.

Warnings are errors, so a compiler warning fails the test step. A clean lint prints that no files were formatted and exits 0. The coverage file is `coverage/<guid>/coverage.cobertura.xml`. `coverage/` is listed in `.gitignore`, so the report is not committed.

How to run the same checks locally

Install the .NET SDK 9. From the repository root, run the three commands the workflow runs. The local lint command restores as well, because a separate restore step is not required on a workstation.

```
dotnet restore max-run.slnx
dotnet format max-run.slnx --verify-no-changes
dotnet test max-run.slnx -c Release --settings coverage.runsettings ^
  --collect:"XPlat Code Coverage" --results-directory ./coverage
```

On macOS or Linux, replace the caret with a backslash. The test command prints the path of `coverage.cobertura.xml`. Open that file to see the line rate and branch rate for `LongestIncreasingRunFinder`.

What you see on GitHub

Open the repository, then the **Actions** tab. Each push and each pull request has a run titled `ci`. The row shows the branch, the commit, and who started it.

Where	What you find
Actions, then the ci run	A green tick when every step exited 0. A red cross when one step failed.
The build job	One log per step. Open Lint or Test with coverage to read the command output.
Artifacts, coverage	A zip of the coverage directory. It is kept with the run and is not added to the repository.
The commit or pull request	The same green tick or red cross, beside the commit message and in the pull request checks.

A failed run does not change the code. Fix the formatting or the test on a new commit and push again. GitHub starts a new run for that commit. The earlier red run stays in the Actions list as history.